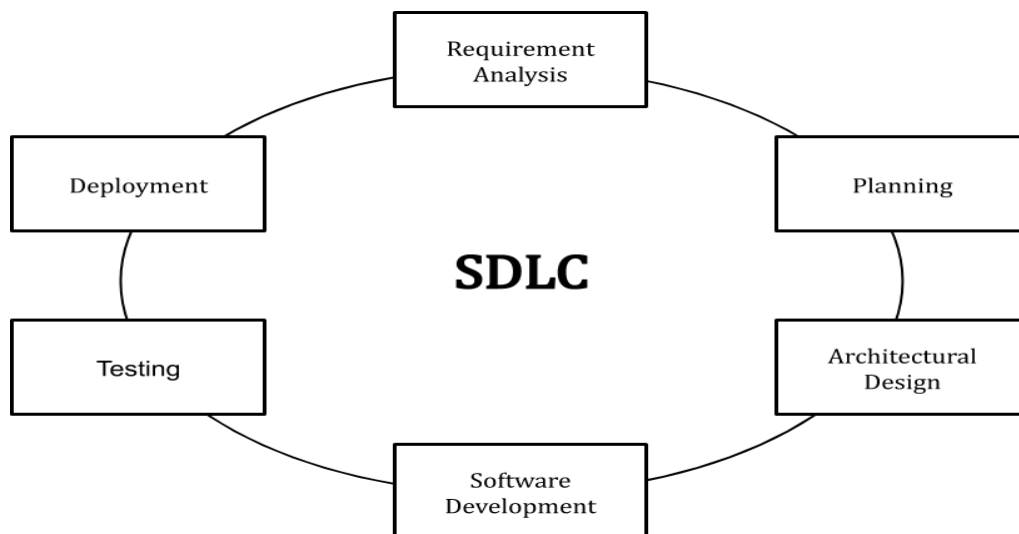


Week-2: SDLC and Process Models

SDLC

- ✓ Software Development Life Cycle (SDLC) is a method (process) by which quality software can be developed in the given time and according to the customer expectations.
- ✓ SDLC ensures quality product.
- ✓ SDLC define a set of guidelines which are to be followed during the development.
- ✓ Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high-quality software.
- ✓ The SDLC aims to produce high-quality software that meets or exceeds customer expectations and reaches completion within times and cost estimates.
- ✓ It is also called as Software Development Process.
- ✓ SDLC is a framework defining tasks performed at each step in the software development process.
- ✓ ISO/IEC 12207 is an international standard for software life-cycle processes. It aims to be the standard that defines all the tasks required for developing and maintaining software.



Stage 1: Planning and Requirement Analysis

Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys, and domain experts in the industry.

Stage 2: Defining Requirements

Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification.

Stage 4: Building or Developing the Product

In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However, this stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization. The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).

Software process model

A software process model is an **abstract representation** of the development process.

A model will define the following:

- ✓ The tasks to be performed
- ✓ The input and output of each task
- ✓ The pre and post conditions for each task
- ✓ The flow and sequence of each task

The goal of a software process model is to provide guidance for controlling and coordinating the tasks to achieve the end product and objectives as effectively as possible.

SDLC Models

There are various software development life cycle models defined and designed which are followed during the software development process. These models are also referred as **Software Development Process Models**.

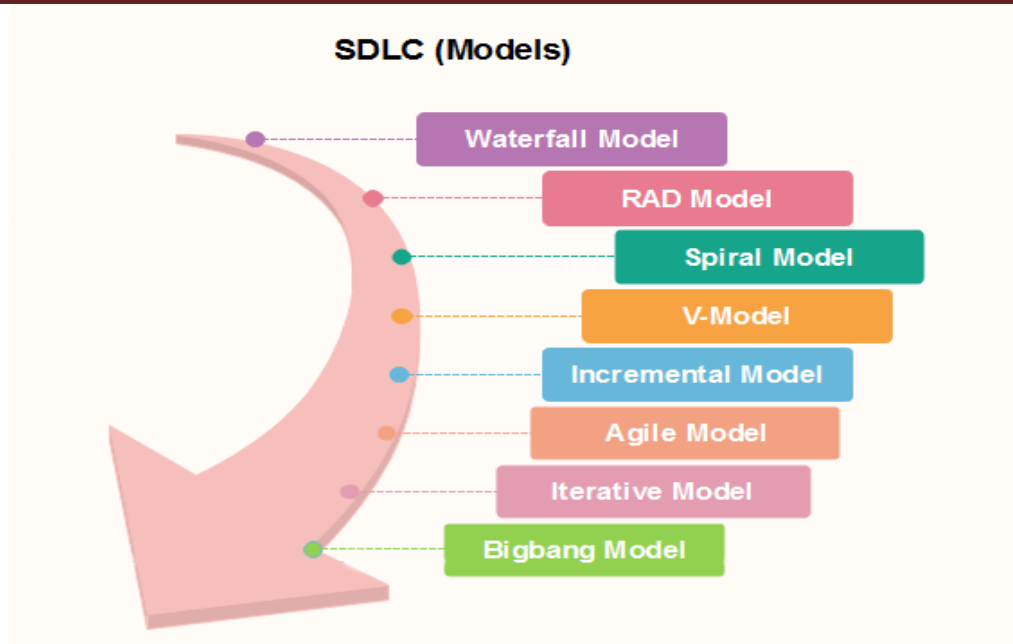
Following are the most important and popular SDLC models followed in the industry –

1. Waterfall Model
2. Iterative Model
3. Spiral Model
4. V-Model
5. Big Bang Model
6. Agile model

Software Development life cycle (SDLC) is a spiritual model used in project management that defines the stages include in an information system development project, from an initial feasibility study to the maintenance of the completed application.

There are different software development life cycle models specify and design, which are followed during the software development phase. These models are also called "**Software Development Process Models**." Each process model follows a series of phase unique to its type to ensure success in the step of software development.

Here, are some important phases of SDLC life cycle:



How to choose process model?

Factors in choosing a software process

Choosing the right software process model for your project can be difficult. If you know your requirements well, it will be easier to select a model that best matches your needs. You need to keep the following factors in mind when selecting your software process model:

Project requirements

Before you choose a model, take some time to go through the project requirements and clarify them alongside your organization's or team's expectations. Will the user need to specify requirements in detail after each iterative session? Will the requirements *change* during the development process?

Project size

Consider the size of the project you will be working on. Larger projects mean bigger teams, so you'll need more extensive and elaborate project management plans.

Project complexity

Complex projects may not have clear requirements. The requirements may change often, and the cost of delay is high. Ask yourself if the project requires constant monitoring or feedback from the client.

Cost of delay

Is the project highly time-bound with a huge cost of delay, or are the timelines flexible?

Customer involvement

Do you need to consult the customers during the process? Does the user need to participate in all phases?

Familiarity with technology

This involves the developers' knowledge and experience with the project domain, software tools, language, and methods needed for development.

Project resources

This involves the amount and availability of funds, staff, and other resources.

How do you choose the appropriate process model?

The basic characteristics required to select the process model are the **project type and associated risks, requirements of the project, and the users**. One of the key features of selecting a process model is to understand the project in terms of size, complexity, funds available, and so on.

Defined Processes

- ✓ Every piece of work is completely understood
- ✓ Defined process can be started and allowed to run with same results every time
- ✓ They provide repeatability and predictability

Empirical Processes

- ✓ Expects the unexpected
- ✓ Because the processes are imperfectly defined, generate unpredictable and unrepeatable output
- ✓ Control is exercised thru inspection and adaptation

Comparison between a defined process and an empirical process

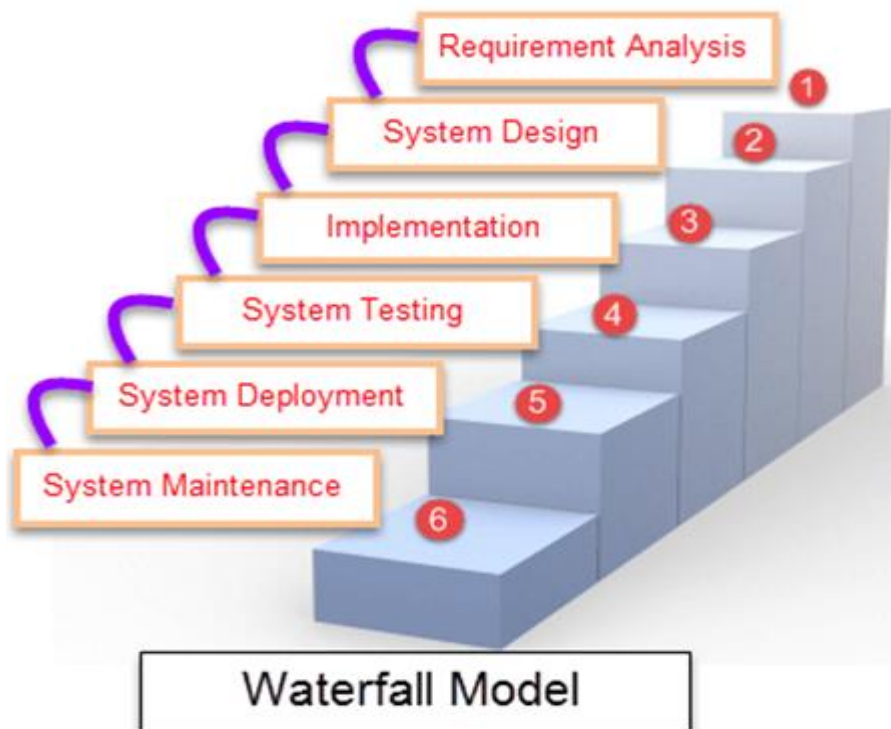
Defined Process	Empirical Process
The process is planned in detail before the project begins.	The planning takes place as the team progresses with the project.
The entire project is completed within a single cycle which may take months.	The entire project is completed in short cycles where it is reviewed after each cycle.
The project manager knows most of the information of the project.	Everyone involved in the project has an equal right to know about the details of the project.
There is a command and control type of leadership among the manager and the developers.	The Scrum or Agile team is self-organizing and have the liberty to ask questions and share their opinions.
The accountability of the project is on the project manager.	The accountability of the project is on everyone who is creating and delivering the product.
The time to market and return on investment takes longer.	The time to market and return on investment takes a shorter time.
The developers are told how and what they have to create and there is no scope of creativity or freedom.	The developers are given an idea such as a user story where they could employ any techniques or software and deliver the result according to their judgment.

Traditional process models:

- ✓ The traditional method **uses a linear approach, where the stages of the software development process must be completed in sequential order.**
- ✓ This means that a stage must be completed before the next one begins.
- ✓ These stages usually comprise the following: Requirements gathering and documentation.
- ✓ It is basically used when the security and many other factors of the software are not much important.
- ✓ It is used by freshers in order to develop the software.

Waterfall Model

- ✓ **Waterfall Model** is a sequential model that divides software development into pre-defined phases.
- ✓ Each phase must be completed before the next phase can begin with no overlap between the phases.
- ✓ Each phase is designed for performing specific activity during the SDLC phase. It was introduced in 1970 by Winston Royce



Different phases	Activities performed in each stage
Requirement Gathering stage	• During this phase, detailed requirements of the software system to be developed are gathered from client
Design Stage	• Plan the programming language, for Example Java, PHP, .net • or database like Oracle, MySQL, etc. • Or other high-level technical details of the project
Built Stage	• After design stage, it is built stage, that is nothing but coding the software
Test Stage	• In this phase, you test the software to verify that it is built as per the specifications given by the client.
Deployment stage	• Deploy the application in the respective environment
Maintenance stage	• Once your system is ready to use, you may later require change the code as per customer request

Waterfall Methodology can be used when:

- Requirements are not changing frequently
- Application is not complicated and big
- Project is short
- Requirement is clear
- Environment is stable
- Technology and tools used are not dynamic and is stable
- Resources are available and trained

Advantages:

These are some advantages of Waterfall Model.

1. It is simple and easy to understand and use.
2. It is easy to manage.
3. It works well for smaller and low budget projects where requirements are very well understood.
4. Clearly defined stages and well understood.
5. It is easy to arrange tasks.
6. Process and results are well documented.

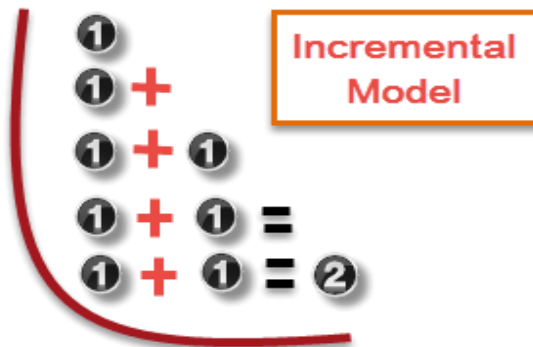
Disadvantages:

These are some disadvantages of Waterfall Model.

1. It is difficult to measure progress within stages.
2. Poor model for long and ongoing projects.
3. No working software is produced until late during the life cycle.
4. High amounts of risk and uncertainty.
5. Not a good model for long and object oriented projects.
6. Cannot accommodate changing requirements.

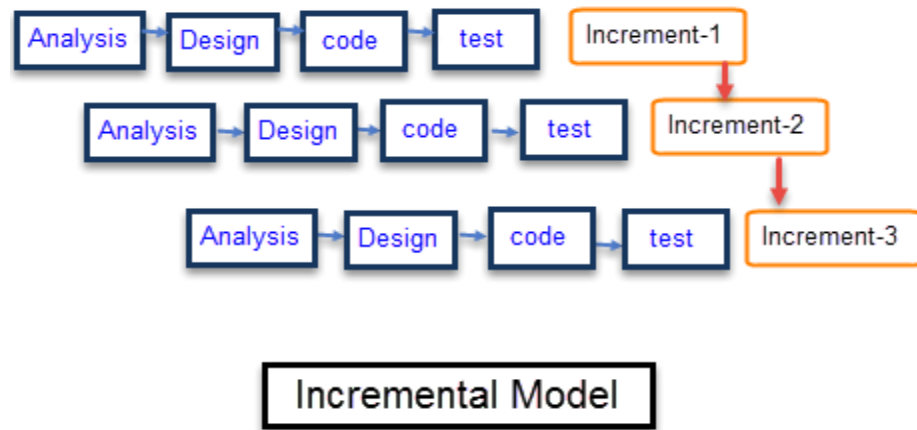
Incremental Model

- ✓ Incremental Model is a process of software development where requirements are broken down into multiple standalone modules of software development cycle.
- ✓ Incremental development is done in steps from analysis design, implementation, testing/verification, maintenance.



- ✓ Each iteration passes through the **requirements, design, coding and testing phases**.

- ✓ And each subsequent release of the system adds function to the previous release until all designed functionality has been implemented.



- ✓ The system is put into production when the first increment is delivered.
- ✓ The first increment is often a core product where the basic requirements are addressed, and supplementary features are added in the next increments.
- ✓ Once the core product is analyzed by the client, there is plan development for the next increment.

Incremental Phases	Activities performed in incremental phases
Requirement Analysis	Requirement and specification of the software are collected
Design	Some high-end function are designed during this stage
Code	Coding of software is done during this stage
Test	Once the system is deployed, it goes through the testing phase

Advantages of incremental Model

1. The software will be generated quickly during the software life cycle
2. It is flexible and less expensive to change requirements and scope
3. Throughout the development stages changes can be done
4. This model is less costly compared to others
5. A customer can respond to each building
6. Errors are easy to be identified.

Disadvantages of incremental Model

1. It requires a good planning designing
2. Problems might cause due to system architecture as such not all requirements collected up front for the entire software lifecycle.
3. Each iteration phase is rigid and does not overlap each other
4. Rectifying a problem in one unit requires correction in all the units and consumes a lot of time

Agile Process:

The Agile methodology is **a way to manage a project by breaking it up into several phases**. It involves constant collaboration with stakeholders and continuous improvement at every stage. Once the work begins, teams cycle through a process of planning, executing, and evaluating.

The workflow of an Agile iteration will typically consist of five steps:

1. Plan requirements.
2. Develop the product.
3. Test software.
4. Deliver iteration.
5. Incorporate feedback.

Agile manifesto

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver it all at once.

The Agile approach to software development is defined by its commitment to creating software incrementally or in regular increases and stages. The approach offers users new versions, or releases, of software following brief periods of work. Those brief periods of work are often called [sprints](#).

The four core values of Agile software development as stated by the Agile Manifesto are:

- Individuals and interactions over processes and tools;
- Working software over comprehensive documentation;
- Customer collaboration over contract negotiation; and
- Responding to change over following a plan.

Following are the Agile Manifesto principles –

1. **Individuals and interactions** – In Agile development, self-organization and motivation are important, as are interactions like co-location and pair programming.
2. **Working software** – Demo working software is considered the best means of communication with the customers to understand their requirements, instead of just depending on documentation.
3. **Customer collaboration** – As the requirements cannot be gathered completely in the beginning of the project due to various factors, continuous customer interaction is very important to get proper product requirements.
4. **Responding to change** – Agile Development is focused on quick responses to change and continuous development.

The 12 principles of Agile

The Agile Manifesto also outlined 12 core principles for the development process. They are as follows:

1. Satisfy customers through early and continuous delivery of valuable work.
2. Break big work down into smaller tasks that can be completed quickly.

3. Recognize that the best work emerges from self-organized teams.
4. Provide motivated individuals with the environment and support they need and trust them to get the job done.
5. Create processes that promote sustainable efforts.
6. Maintain a constant pace for completed work.
7. Welcome changing requirements, even late in a project.
8. Assemble the project team and business owners on a daily basis throughout the project.
9. Have the team reflect at regular intervals on how to become more effective, then tune and adjust behavior accordingly.
10. Measure progress by the amount of completed work.
11. Continually seek excellence.
12. Harness change for a competitive advantage.

Best Practices for Agile Teams

1. Collaborate with the customer

The customer is satisfied when requirements are fulfilled, expectations are met, and wants and needs are gratified.

2. Work together daily

Teamwork, contribute to successful projects when they collaborate as a cohesive unit.

- ✓ Communication
- ✓ Coordination
- ✓ Balance of team member contributions
- ✓ Mutual support
- ✓ Effort
- ✓ Cohesion

3. Build projects around motivated individuals

It takes motivation to push through an intense development cycle and get the work done right. Agile teams are passionate about their work, focused on the team goal, and supportive of each other. When there is trust and respect among peers, Agile teams establish a rhythm to their work that is fast-paced and predictable.

4. Convey information face-to-face

Whether working through a knotty problem with a teammate or reporting on the day's accomplishments at a daily meeting, Agile team members prefer face-to-face communication. Information lost in a full email box or voice message queue slows or blocks progress. The daily meeting is one time the entire team connects to find out if there are any issues that could cause delays. Face-to-face conversation opens channels and builds trust, making the Agile method sustainable.

5. Form self-organizing teams

Choose how they will execute the work, and who will do what. They divide the work into increments that can be completed within each iteration, and into tasks that can be completed each day. Management does not assign tasks or look over their shoulders. The team is entrusted with making the right decisions. For this arrangement to work, each team member has to be confident in their work and commit to pushing through the most difficult, frustrating blocks.

6. Reflect on how teams can become more effective

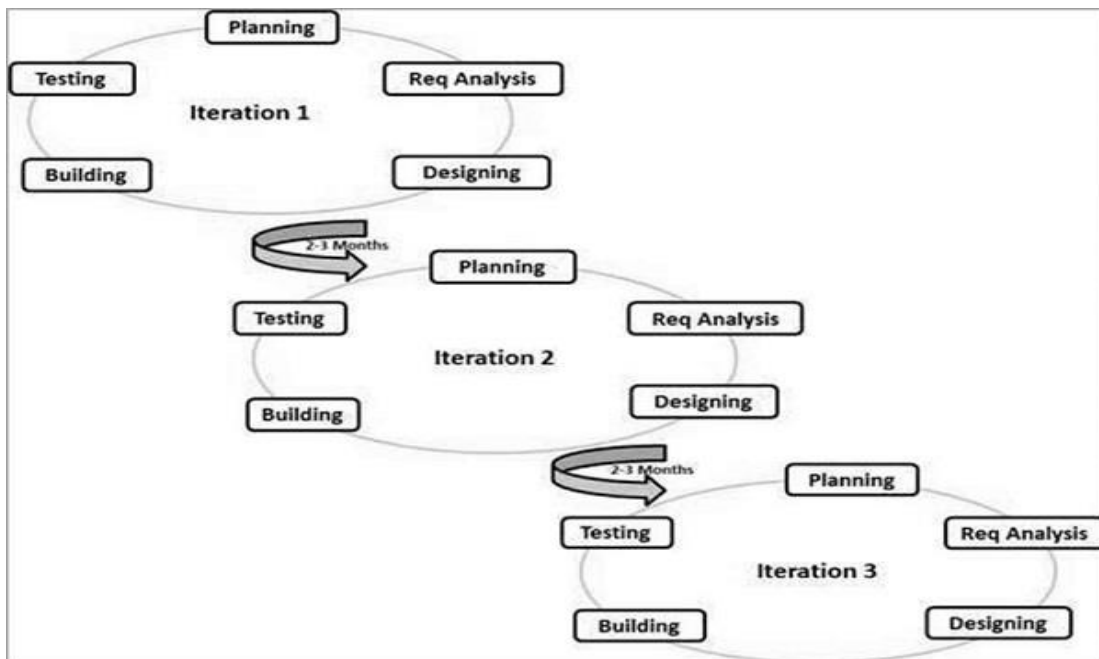
Agile teams routinely examine their performance and look for ways to do better. In fact, they are dedicated to continuous improvement. As individuals and as a group, teams are open to mentoring and coaching but, as one Agile coach notes, they don't respond well to command and control behavior from management.

Agile model

Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. Every iteration involves cross functional teams working simultaneously on various areas like

1. Planning
2. Requirements Analysis
3. Design
4. Coding
5. Unit Testing and
6. Acceptance Testing.

At the end of the iteration, a working product is displayed to the customer and important stakeholders.



The most popular Agile methods include Rational Unified Process (1994), Scrum (1995), Crystal Clear, Extreme Programming (1996), Adaptive Software Development, Feature

Driven Development, and Dynamic Systems Development Method (DSDM) (1995). These are now collectively referred to as **Agile Methodologies**, after the Agile Manifesto was published in 2001.

The advantages of the Agile Model are as follows

1. Is a very realistic approach to software development?
2. Promotes teamwork and cross training.
3. Functionality can be developed rapidly and demonstrated.
4. Resource requirements are minimum.
5. Suitable for fixed or changing requirements
6. Delivers early partial working solutions.
7. Good model for environments that change steadily.
8. Minimal rules, documentation easily employed.
9. Enables concurrent development and delivery within an overall planned context.
10. Little or no planning required.
11. Easy to manage.
12. Gives flexibility to developers.

The disadvantages of the Agile Model are as follows

1. Not suitable for handling complex dependencies.
2. More risk of sustainability, maintainability and extensibility.
3. An overall plan, an agile leader and agile PM practice is a must without which it will not work.
4. Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
5. Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
6. There is a very high individual dependency, since there is minimum documentation generated.
7. Transfer of technology to new team members may be quite challenging due to lack of documentation.

Three Paradigm Shifts of Agile

1. Triple-Constraints of Project Management (aka – The Iron Triangle).

2. Traditional vs Emergent Requirements.
3. Project Managers vs Scrum Masters.

Triple-Constraints of Project Management (aka – The Iron Triangle)

An Agile approach to a project is much more thoughtful about the cadence of the project. How long are our iterations (we fix that), who is on the dedicated cross-functional team (fixed run-rate budget), and what is the vision of what we are trying to achieve. We then pick a release planning approach and move forward keeping the team and the sprint as sacred as possible. That leaves us the ability to focus our cognitive processing ability not on managing change requests, re-baselining project plans, or updating requirements documents – but instead we are focusing on delivering the right thing.

Traditional vs Emergent Requirements

In traditional projects all the requirements of a project are specified upfront at a very precise level of detail. And because our projects are set up for failure having fixed the scope, cost, and time – when we start coming close to the end of the timeline or the end of the budget we inevitably have to de-scope something so we leave inventory of the table and deliver less than what we originally specified upfront. This is a significant waste within software development and is a key driver of why there are trust issues between business and IT groups. Instead of specifying everything at great detail upfront and delivering something less, an Agile framework turns that upside down. We want to focus first on a very well defined Vision, decompose that into Themes (large chunks of functionality), and based on the priority of those Themes start elaborating more specific detail of the features, and then the highest priority features into stories, and the highest priority stories into technical designs and ultimately working software.

Project Managers vs Scrum Masters:

Agile, on the other hand doesn't require a Project Management role. Instead we look to the role of ScrumMaster. And Scrum Masters work for teams. Teams that are self-organizing and cross-functional. Teams that are in control of their own destiny and ultimately their own ability to succeed or fail. Instead of command and control, Scrum Masters act as Servant Leaders that insulate the team from distractions, remove impediments, and provide the tools and resources needed so the team can maximize success.